PHASE 3 OF 8

Project Design

Architecture, data flow and interfaces of LegalEase

Project: LegalEase – AI-Powered Legal Document Generator
Presented by: Ganesh Prabhu, Gopinath, Hariharan, Gowtham

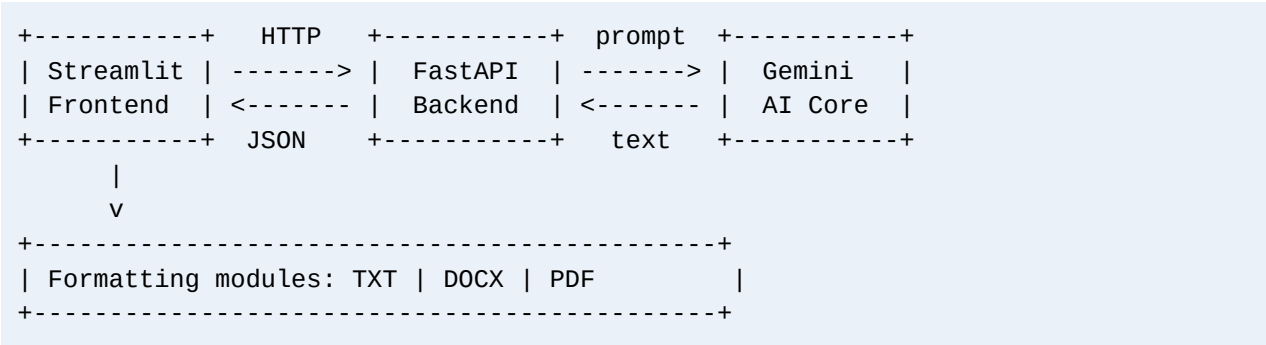
3.1 Purpose of this Phase

The Project Design phase defines how LegalEase is structured so that the requirements can be implemented cleanly. It describes the major components, how data moves between them, the interface between frontend and backend, and the main design decisions taken by the team.

3.2 System Architecture

The system is divided into four major components. Each has a single, well-defined responsibility, which keeps the application easy to build, test and modify.

Component	Technology	Responsibility
Frontend	Streamlit	Collects user input, shows the generated preview, allows editing and provides download buttons
Backend	FastAPI (served by Uvicorn)	Receives requests, validates them and returns responses through an API
AI Core	Google Gemini	Turns the structured inputs into legal-document text
Formatting Modules	python-docx, FPDF, Pillow	Produce TXT, DOCX and PDF output including headings, logo and footer



3.3 Data Flow

The end-to-end flow of information is: **User Input** → **Streamlit** → **FastAPI /generate** → **Gemini AI** → **Generated Text** → **Preview/Edit** → **TXT/DOCX/PDF Export.** In more detail:

1. The user fills in the form and clicks *Generate Document*.
2. Streamlit packages the inputs and sends them to the FastAPI **/generate** endpoint.

3. FastAPI validates the request with a Pydantic model and passes the data to the AI core.
4. The AI core builds a prompt from the inputs and calls Gemini.
5. The generated text is returned through the API to Streamlit.
6. Streamlit displays the text in an editable preview area.
7. On download, the (possibly edited) text is passed to the selected formatter and returned as a file.

3.4 API Design

The backend exposes a single primary endpoint. Its contract is intentionally small so that the frontend and backend remain loosely coupled.

Item	Details
Endpoint	POST /generate
Request body (JSON)	document type, parties, terms and conditions, effective date, jurisdiction
Validation	Pydantic model rejects missing or wrongly typed fields with a validation error
Success response	Generated document text
Error response	Clear error message when validation or AI generation fails

3.5 User Interface Design

The Streamlit interface is a single page divided into three logical areas:

- **Input area:** selection box for document type, fields for parties, a text area for terms, a date picker and a jurisdiction field.
- **Preview area:** shows the generated document and lets the user edit it in place.
- **Download area:** buttons to export the current text as TXT, DOCX or PDF.

The layout follows the natural order of the task – input, generate, review, export – so the user always knows the next step.

3.6 Prompt Design for Gemini

The quality of the draft depends on the prompt. The AI core builds the prompt from the user's inputs and instructs the model to produce a clearly structured document, for example: a title, the effective date, identification of the parties, numbered clauses covering the supplied terms, the governing jurisdiction, and a signature section. Keeping prompt construction inside one module allows the wording to be improved without touching the API or the interface.

3.7 Formatting Module Design

Format	Library	Design notes
TXT	Python file handling	Plain text, maximum compatibility
DOCX	python-docx	Headings, paragraphs and terms as styled Word elements; optional logo and footer
PDF	FPDF	Formatted layout with heading, body text, logo and footer; images processed with Pillow

3.8 Key Design Decisions

- **Separate frontend and backend** so the AI logic is reachable through an API and can be tested independently of the interface.
- **Editable preview before export** because AI drafts are rarely final.
- **Environment-file configuration** (python-dotenv) to keep the API key out of source code.
- **Modular formatters** so a new export format can be added without altering existing ones.

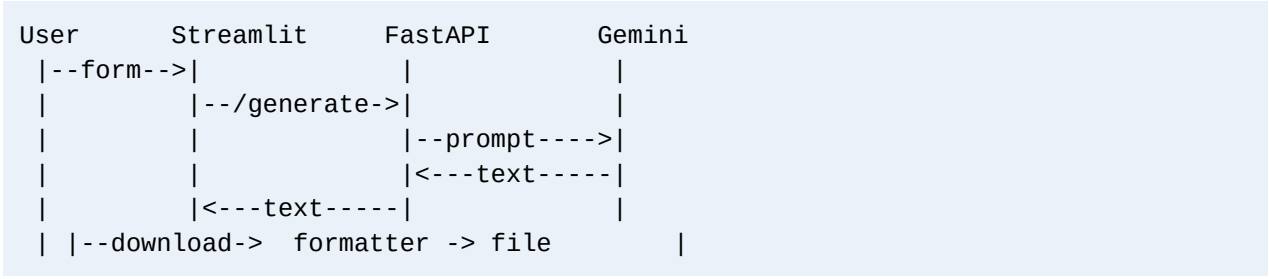
3.9 Error Handling Design

Invalid input is rejected by Pydantic validation at the API boundary. If Gemini cannot be reached or returns an error, the backend returns a readable message and the frontend shows it to the user instead of failing silently.

3.10 Module and Class Responsibilities

Module	Responsibility	Interacts with
Frontend module	Renders the form, calls the API, shows preview and download buttons	FastAPI backend, formatters
API module	Defines the /generate route and the Pydantic request model	AI core
AI core module	Builds the prompt, calls Gemini and returns the text	Gemini service
TXT formatter	Writes plain text output	Frontend
DOCX formatter	Creates the Word document using python-docx	Frontend, Pillow
PDF formatter	Creates the PDF document using FPDF	Frontend, Pillow

3.11 Sequence of Interaction



3.12 Configuration Design

Runtime settings such as the Gemini API key and the backend address are loaded from an environment file using python-dotenv. This keeps secrets out of the code base, lets each team member use their own key, and allows the backend address to change without editing the interface code.

3.13 Design for Editing and State

After generation, the text is held in the interface session and shown in an editable text area. When the user changes the text, the export buttons always use the latest edited version, so the downloaded file matches what is on screen. Regenerating replaces the current text with a new draft.

3.14 Design Constraints

- Output quality depends on the model and the wording of the prompt.
- Generation time depends on network and model response speed.
- The design targets a single user running the application locally.